

Lösung

Tutoraufgabe 6.1

- (a) Entscheiden Sie die folgenden PKP-Instanzen:

$$K_1 = \left\{ \left[\frac{a}{abb} \right], \left[\frac{ab}{b} \right], \left[\frac{ab}{ba} \right], \left[\frac{aba}{ba} \right] \right\}$$
$$K_2 = \left\{ \left[\frac{a}{b} \right], \left[\frac{b}{aa} \right], \left[\frac{b}{ba} \right], \left[\frac{bb}{ba} \right], \left[\frac{aab}{b} \right] \right\}$$

Lösung:

K_1 ist eine Nein-Instanz. Einzig mögliches Start-Domino ist das 1. Domino. Danach müsste ein Domino folgen, welches oben mit b beginnt. Jedoch gibt es kein solches Domino.

K_2 ist eine Ja-Instanz. Zum Beispiel ergeben das 3. 1. 2. 5. Domino oben und unten das Wort $babaab$.

- (b) Beweisen oder widerlegen Sie: Das PKP über dem Alphabet $\{0, 1\}$ ist unentscheidbar.

Lösung:

Wir reduzieren das PKP auf as PKP über dem Alphabet $\{0, 1\}$. Da das PKP unentscheidbar ist, folgt, dass auch PKP über dem Alphabet $\{0, 1\}$ unentscheidbar ist.

Konstruktion: Ersetze das Alphabet Σ durch $\{0, 1\}$. Sei $|\Sigma| = m$ und $\Sigma = \{a_1, \dots, a_m\}$. Für alle Buchstaben $a_i \in \Sigma, i \in \{1, \dots, m\}$ ersetze alle Vorkommen auf Dominos von a_i durch $10^i 1$.

Für die Korrektheit ist zu zeigen, dass eine gegeben PKP-Instanz eine Lösung hat gdw. die konstruierte PKP-Instanz eine Lösung hat.

($\Rightarrow$) Seien die Dominos $D_1, \dots, D_n$ eine PKP Instanz mit Lösung $i_1, \dots, i_s, s \geq 1$. Sei $D'_1, \dots, D'_n$ die Dominos mit ersetzten Buchstaben. Dann sind die Dominos in gleicher Reihenfolge $i_1, \dots, i_s$ auch eine Lösung für die Dominos $D'_1, \dots, D'_n$.

($\Leftarrow$) Seien die konstruierten Dominos $D'_1, \dots, D'_n$ (über dem Alphabet $\{0, 1\}$) eine PKP Instanz mit Lösung $i_1, \dots, i_s, s \geq 1$. Die Dominos $D'_1, \dots, D'_n$ in der Reihenfolge $i_1, \dots, i_s$ ergeben ein Wort $b_1, \dots, b_t, t \in \mathbb{N}$. Seien $D_1, \dots, D_n$ die ursprünglichen Dominos. Die Dominos $D_1, \dots, D_n$ in der Reihenfolge $i_1, \dots, i_s$ ergeben oben ein Wort $c_1 c_2, \dots$. Die Dominos $D_1, \dots, D_n$ in der Reihenfolge $i_1, \dots, i_s$ ergeben unten ein Wort $d_1 d_2, \dots$. Für einen Widerspruch nehmen wir an, dass es eine *kleinste* Position $i \in \mathbb{N}$ gibt mit $c_i \neq d_i$. Sei $c_i = a_k$ und $d_i = a_\ell$. Dann ergeben die Dominos $D'_1 \dots D'_n$ in Reihenfolge $i_1, \dots, i_s$ oben ein Wort mit Prefix $w' 10^k 1$ und unten ein Wort mit Prefix $w' 10^\ell 1$ für ein $w' \in \{0, 1\}^*$. Widerspruch dazu, dass die Dominos $D'_1, \dots, D'_n$ in der Reihenfolge $i_1, \dots, i_s$ oben und unten das gleiche Wort haben. Also ist die Reihenfolge $i_1, \dots, i_s$ eine Lösung für die ursprüngliche PKP-Instanz $D_1, \dots, D_n$.

- (c) Beweisen oder widerlegen Sie: Das PKP über dem unären Alphabet $\{0\}$ ist entscheidbar.

Lösung:

Es gibt eine Lösung, falls (1) es ein Domino D_1 gibt, wo das obere Wort um $x \geq 0$ länger ist als das untere, und es ein Domino D_2 gibt, wo das obere Wort um $y \leq 0$ länger ist als das untere (möglicherweise $D_1 = D_2$). Die Lösung besteht dann aus y mal Domino D_1 gefolgt von x mal Domino D_2 . Sonst, gilt entweder für alle Dominos, dass das obere Wort echt länger ist als das untere, oder es gilt, dass für alle Dominos das obere Wort echt kürzer ist als das obere. Dann gibt es offensichtlich keine Lösung.

PKP über dem unären Alphabet $\{0\}$ ist entscheidbar, da man nur die Eingabe auf (1) überprüfen muss.

Tutoraufgabe 6.2

Die folgenden Beweise enthalten Fehler. Identifizieren Sie diese und geben Sie ggf. an, warum die verwendete Beweistechnik nicht angewendet werden kann.

(a) Sei H_{tot} gegeben durch

$$H_{tot} = \{\langle M \rangle \mid M \text{ hält auf allen Eingaben}\}$$

Wir zeigen, dass $\overline{H_{tot}}$ nicht rekursiv aufzählbar ist durch Reduktion von H_{tot} . Als Funktion verwenden wir dafür die Identität, invertieren jedoch die Ausgabe. Diese Funktion ist offensichtlich berechenbar.

Korrektheit:

- Sei $x \in H_{tot}$. Dann ist nach Definition $f(x) \in \overline{H_{tot}}$.
- Sei $f(x) \in \overline{H_{tot}}$. Dann ist nach Definition $x \in H_{tot}$.

Lösung:

Eine Funktion kann an der Ausgabe des Problems nichts ändern, die angegebene Reduktionsabbildung ist also keine Funktion. Die angegebene Konstruktion ist also eigentlich eine Unterprogrammtechnik, mit der man aber rekursive Aufzählbarkeit nicht zeigen oder widerlegen kann. Das liegt daran, dass die Menge der rekursiv aufzählbaren Sprachen nicht unter Komplement abgeschlossen ist.

(b) Sei $L_1 = \{\langle M \rangle \mid L(M) \text{ ist unendlich}\}$.

Wir benutzen den Satz von Rice um zu zeigen, dass L_1 unentscheidbar ist.

1. Sei

$$S = \{f \in \mathcal{R} \mid L(M) \text{ ist unendlich}\}.$$

2. Dann gilt

$$\begin{aligned} L(S) &= \{\langle M \rangle \mid M \text{ berechnet eine Funktion aus } S\} \\ &= \{\langle M \rangle \mid L(M) \text{ ist unendlich}\} \\ &= L_1. \end{aligned}$$

3. Es gilt $L(S) \neq \emptyset$, da die TM, die immer sofort akzeptiert, in $L(S)$ liegt.

4. Außerdem gilt $L(S) \neq \mathcal{R}$, da die TM, die immer verwirft, nicht in $L(S)$ liegt.

Damit ist L_1 nach dem Satz von Rice unentscheidbar.

Lösung:

L_1 ist zwar unentscheidbar, und der Satz von Rice ist auch anwendbar, die obige Lösung ist jedoch nicht korrekt.

Zu 1. S beschreibt eine Menge von partiellen Funktionen, also keine Turingmaschinen oder Gödelnummern. Man darf also nicht das Verhalten irgendwelcher Turingmaschinen in der Definition von S referenzieren. Eine korrekte Definition von S wäre in diesem Fall z.B.

$$S = \{f \in \mathcal{R} \mid f(w) = 1 \text{ für unendlich viele } w \in \{0, 1\}^*\},$$

oder wenn man es formeller ausdrücken möchte

$$S = \{f \in \mathcal{R} \mid |f^{-1}(1)| = \infty\}.$$

$f^{-1}(1)$ bezeichnet hierbei das Urbild von f von 1, also alle $w \in \{0, 1\}^*$ mit $f(w) = 1$.

Zu 3. Die Bedingung des Satzes von Rice fordert $S \neq \emptyset$. Man muss also auch keine TM, sondern eine Funktion angeben, um zu zeigen, dass $S \neq \emptyset$. Daher verwenden wir auch oft die Formulierung *die berechnete Funktion der TM* Weiterhin sei hier angemerkt, dass es wichtig ist, über Funktionen zu argumentieren, da der Satz von Rice nur anwendbar ist, wenn *alle* oder *keine* der TM, die eine Funktion berechnen, in der betrachteten Sprache liegen.

Zu 4. Hier besteht das gleiche Problem wie bei 3. Zusätzlich gilt aber auch *immer*, dass $L(S) \neq \mathcal{R}$, da $L(S)$ eine Menge von Gödelnummern ist, während $\mathcal{R}$ eine Menge von Funktionen ist.

(c) Sei $L_2 = \{\langle M \rangle \mid M \text{ akzeptiert } \langle M \rangle\}$.

Wir benutzen den Satz von Rice um zu zeigen, dass L_2 unentscheidbar ist.

1. Sei

$$S = \{f \in \mathcal{R} \mid f(w) = 1, \text{ wenn } w = \langle M \rangle \text{ und } M \text{ akzeptiert } \langle M \rangle\}.$$

2. Dann gilt

$$\begin{aligned} L(S) &= \{\langle M \rangle \mid M \text{ berechnet eine Funktion aus } S\} \\ &= \{\langle M \rangle \mid \langle M \rangle \in L(M)\} \\ &= L_2. \end{aligned}$$

3. Es gilt $S \neq \emptyset$, da die Gödelnummer, die alles akzeptiert in S liegt.

4. Außerdem gilt $S \neq \mathcal{R}$, da die Gödelnummer, die immer verwirft, nicht in S liegt.

Damit ist L_3 nach dem Satz von Rice unentscheidbar.

Lösung:

L_2 ist zwar unentscheidbar, aber man kann dies nicht aus dem Satz von Rice folgern. In der Definition von S wird direkt Bezug auf das Verhalten einer TM genommen, was so nicht möglich ist, da S eine Menge von partiell berechenbaren Funktionen ist. Wie schon in der vorigen Aufgabe angemerkt, ist der Satz von Rice nur anwendbar, wenn *alle* oder *keine* der TM, die eine Funktion berechnen, in der betrachteten Sprache liegen. Wir betrachten nun folgende Funktion:

$$f_{1000000}(w) = \begin{cases} 1, & \text{wenn } |w| \leq 1000000 \\ 0, & \text{wenn } |w| > 1000000 \end{cases}$$

Es ist leicht zu sehen, dass es eine TM M_a gibt, deren Gödelnummer weniger als 1000000 Zeichen hat, die $f_{1000000}$ berechnet: Die Turingmaschine von Blatt 1, die den Logarithmus des Eingabewortes berechnet, lässt sich leicht so modifizieren, dass sie anfangs 1000000 getrennt mit einem # links von dem Eingabewort schreibt, und dies als Zähler benutzt, der für jedes Zeichen in der Eingabe dekrementiert wird. Unsere TM hat also weniger als 30 Zustände, und ein Bandalphabet mit 4 Zeichen. Die Kodierung jedes Übergangs so wie auf Folie 2/22 benötigt also weniger als 80 Zeichen. Insgesamt haben wir damit $30 \cdot 4 \cdot 80 = 9600$ Zeichen für alle Übergänge, plus ein paar Trennzeichen, insgesamt also deutlich weniger als 1000000. Damit gilt $f_{1000000}(\langle M_a \rangle) = 1$ und M_a akzeptiert also ihre eigene Gödelnummer.

Wir betrachten nun eine TM M_b , die eine Kopie von M_a ist, der aber noch 1000000 Zustände hinzugefügt wurden (mit beliebigen Übergängen untereinander), die jedoch nicht von den Zuständen von M_a erreicht werden. Nach der Kodierung aus der Vorlesung benötigt nun die Kodierung eines Übergangs, der $q_{1000000}$ beinhaltet, mehr als 1000000 Zeichen. Also ist die Gödelnummer von M_b auf jeden Fall länger als 1000000 Zeichen. Damit gilt $f_{1000000}(\langle M_b \rangle) = 0$ und M_b verwirft also ihre eigene Gödelnummer.

Es gilt also insgesamt, dass $\langle M_a \rangle \in L_2$ und $\langle M_b \rangle \notin L_2$ und weiterhin dass $f_{M_a} = f_{M_b} = f_{1000000}$. Wenn wir nun ein S betrachten mit $f_{1000000} \in S$ gilt, dass $\langle M_b \rangle \in L(S)$, da M_b die Funktion $f_{1000000}$ berechnet. Also gilt $L(S) \neq L_2$, da $\langle M_b \rangle \notin L_2$. Wenn wir nun ein S betrachten mit $f_{1000000} \notin S$ gilt, dass $\langle M_a \rangle \notin L(S)$, da M_a die Funktion $f_{1000000}$ berechnet. Also gilt $L(S) \neq L_2$, da $\langle M_a \rangle \in L_2$.

Damit kann es kein S geben, sodass $L(S) = L_2$, und der Satz von Rice ist nicht anwendbar.

Weiterhin wird in 3. und 4. von Gödelnummern gesprochen, S und $\mathcal{R}$ sind jedoch Mengen von Funktionen.

(d) Sei L_3 gegeben durch

$$L_3 = \{\langle M \rangle \mid M \text{ akzeptiert die Eingabe 1}\}$$

Wir benutzen Unterprogrammtechnik, um zu zeigen, dass L_3 nicht entscheidbar ist. Angenommen, es gibt eine TM M_3 , die L_3 entscheidet. Wir nutzen diese, um eine TM M_H zu konstruieren, die das Halteproblem entscheidet.

Gegeben $\langle M \rangle w$ konstruieren wir eine TM M' , die sich wie folgt verhält:

- Wenn M' als Eingabe 1 bekommt, simuliert M auf w . Wenn die Simulation hält, verwirft M' , sonst akzeptiert M' .
- Bei Eingaben $w \neq 1$ verwirft M' direkt.

Unsere TM M_H konstruiert bei Eingabe $\langle M \rangle w$ die TM M' und ruft M_3 mit $\langle M' \rangle$ als Eingabe auf. Wenn M_3 verwirft, akzeptiert M_H , und wenn M_3 akzeptiert, verwirft M_H . Wenn M_H keine korrekt formatierte Eingabe erhält, verwirft sie sofort.

Alle Schritte der Konstruktion sind berechenbar.

Korrektheit:

($\Rightarrow$) Sei $\langle M \rangle w \in H$. Dann hält die Simulation von M auf w und M' verwirft bei Eingabe 1. Damit verwirft M_3 die Eingabe $\langle M' \rangle$. Dann akzeptiert M_H .

($\Leftarrow$) Wenn w' nicht die Form $\langle M \rangle w$ hat, verwirft M_H . Sei also $\langle M \rangle w \notin H$. Dann hält die Simulation von M auf w nicht, und M' akzeptiert bei Eingabe 1. Damit akzeptiert M_3 die Eingabe $\langle M' \rangle$. Dann verwirft M_H .

Lösung:

Die Sprache L_3 ist unentscheidbar, und dies lässt sich auch durch Unterprogrammtechnik zeigen. Die gegebene Konstruktion ist jedoch falsch, da die TM M' selber prüft, ob M auf w hält. Für den Fall, dass M auf w tatsächlich hält, ist die angegebene Konstruktion korrekt, da M' dies feststellen kann. Das eine Simulation nicht terminiert, ist jedoch i.A. nicht erkennbar, denn dann wäre $\overline{H}$ rekursiv aufzählbar, was es nach VL nicht ist. Damit kann M' nicht akzeptieren „sobald“ M auf w nicht hält. Die Konstruktion ist also nicht berechenbar.

Anmerkung: Für jedes feste w gibt es eine TM M' , die sich immer korrekt verhält (entweder die TM, die immer verwirft, oder die, die immer akzeptiert). Es ist jedoch nicht entscheidbar, welche davon die richtige ist, da man damit das Halteproblem lösen würde.

Korrekte Konstruktion: Bei Eingabe 1 simuliert M' die TM M auf w , und akzeptiert, sobald die Simulation hält. Bei anderen Eingaben verhält sich M' beliebig.

Unsere TM M_H konstruiert bei Eingabe $\langle M \rangle w$ die TM M' und ruft M_3 mit $\langle M' \rangle$ als Eingabe auf. Wenn M_H akzeptiert, wenn M_3 akzeptiert, und verwirft wenn M_3 verwirft. Wenn M_H keine korrekt formatierte Eingabe erhält, verwirft sie sofort.

Die Konstruktion von M' gegeben $\langle M \rangle w$ ist berechenbar.

Korrektheit:

($\Rightarrow$) Sei $\langle M \rangle w \in H$. Dann hält die Simulation von M auf w und M' akzeptiert bei Eingabe 1. Damit akzeptiert M_3 die Eingabe $\langle M' \rangle$. Dann akzeptiert M_H .

($\Leftarrow$) Wenn w' nicht die Form $\langle M \rangle w$ hat, verwirft M_H . Sei also $\langle M \rangle w \notin H$. Dann hält die Simulation von M auf w nicht, und M' hält nicht auf Eingabe 1. Damit verwirft M_3 die Eingabe $\langle M' \rangle$. Dann verwirft M_H .